

EVA-PRD

# PRODUCT REQUIREMENTS DOCUMENT (PRD)

## Product Name

EVA — Autonomous Data Analysis Assistant

---

## 1. Product Vision

EVA is a personal, self-operating data analyst that transforms messy raw data into understandable insights and usable predictions without requiring users to know statistics, programming, or machine learning.

The product's purpose is simple:

A human should be able to upload a dataset and understand what it means, what will happen next, and what decision to take — without learning data science.

EVA is not a “tool”.

EVA behaves like a **junior analyst + statistician + business consultant combined into one assistant**.

---

## 2. Problem Statement

Today, working with data has 3 major barriers:

### A. Most time is not spent on intelligence

Users spend majority effort on:

- cleaning files
- fixing missing values
- formatting columns
- trial-and-error analysis

Instead of insight, they do maintenance work.  
The presentation shows data cleaning & EDA consume most effort .

---

## **B. Data science knowledge barrier**

To answer even simple questions like:

- “Why are sales dropping?”
- “Which patients are risky?”
- “Which customers will churn?”

A user must understand:

- statistics
- feature selection
- modeling
- evaluation

Most students, startups, analysts, and business owners cannot.

---

## **C. Privacy & trust barrier**

Users hesitate to use external platforms because:

- business data is confidential
- medical data is sensitive
- academic datasets are restricted

They want intelligence **without giving away data**.

---

# **3. Target Users**

## **Primary Users**

1. Students learning data analysis
2. Freelancers working with client datasets
3. Analysts who are not programmers
4. Small businesses with spreadsheets

## Secondary Users

1. Healthcare researchers
  2. Educators & professors
  3. Non-technical founders
  4. Consultants
- 

## 4. Core Value Proposition

EVA offers **decision intelligence**, not just analytics.

It does three things humans actually want:

1. Understand the data
  2. Predict future outcomes
  3. Recommend actions
- 

## 5. Product Goals

### Business Goals

- Remove need to hire a data analyst for basic analysis
- Enable non-technical users to make data-driven decisions
- Become the default “first step” after collecting data

### User Goals

A user should be able to:

- upload a dataset
  - ask questions in plain English
  - receive conclusions they can act on
- 

## 6. Key User Stories

### Student

“I have a dataset for my project. I don’t know what analysis to do. Tell me what is important and help me explain it.”

### **Freelancer**

“My client gave me a messy CSV file. Clean it and tell me the main findings.”

### **Business Owner**

“I have customer data. Which customers will leave and why?”

### **Researcher**

“Find patterns I might have missed.”

---

## **7. Functional Requirements**

### **7.1 Dataset Understanding**

When a dataset is uploaded, EVA must:

- identify column meanings
- detect numerical vs categorical data
- recognize dates, IDs, targets
- detect errors or anomalies

EVA should automatically explain:

“This dataset appears to track customer behavior over time.”

---

### **7.2 Data Cleaning (Autonomous)**

EVA automatically:

- handles missing values
- corrects inconsistent entries
- removes duplicates
- fixes formatting
- flags suspicious data

User should **not need to manually prepare data.**

---

## 7.3 Exploratory Insights

EVA generates human explanations such as:

- important factors
- unusual behavior
- correlations
- segments/groups

Output must be **story-like**, not statistical jargon.

Example:

“Age and purchase frequency strongly influence customer retention.”

---

## 7.4 Predictive Reasoning

EVA determines if prediction is possible and:

- predicts future values
- estimates risks
- ranks importance of factors

It should also explain:

why the prediction occurs.

---

## 7.5 Decision Guidance

The product must go beyond prediction.

It must answer:

**“What should I do now?”**

Example outputs:

- Which customers to contact
- Which patients are high-risk
- Which features matter most
- What change improves results

---

## 7.6 Conversational Interaction

Users interact with EVA like a human analyst:

Users can ask:

- “Why did sales drop?”
- “What variable matters most?”
- “Explain in simple words”
- “Give summary for report”

EVA responds conversationally.

---

## 7.7 Explainability

Every conclusion must include:

- reasoning
- confidence
- interpretation in simple language

EVA should teach while analyzing.

---

## 7.8 Output Generation

EVA produces:

- report summary
- findings explanation
- ready-to-present insights
- conclusions

The user should be able to directly present results.

---

# 8. Non-Functional Requirements

## Privacy

- User data must never leave the user's control
- No external sharing
- No hidden processing

## **Transparency**

EVA must explain:

- what it did
- why it did it
- how confident it is

## **Usability**

A 10th-grade student should be able to use it without training.

## **Speed**

Insight generation should feel immediate and interactive.

---

# **9. Product Behavior Flow**

1. User uploads dataset
2. EVA studies structure
3. EVA cleans issues automatically
4. EVA identifies patterns
5. EVA predicts outcomes
6. EVA explains results
7. EVA suggests decisions
8. User asks questions
9. EVA clarifies and refines

This matches the “raw file → analysis → visualization → training → output” journey illustrated in the deck .

---

# **10. Success Metrics**

## **Adoption Metrics**

- Users who successfully analyze dataset on first attempt



- Time taken to reach first insight

## Outcome Metrics

- Users who understand their data after upload
- Users able to present results

## Retention Metrics

- Returning users with new datasets
  - Usage across multiple domains
- 

# 11. Product Differentiation

EVA is different because:

Typical analytics tools → help you analyze

EVA → **does the analysis for you**

Typical AI tools → answer questions

EVA → **answers questions about your data**

Typical ML platforms → require expertise

EVA → **removes expertise requirement**

---

# 12. Future Expansion (Logical Direction)

Potential evolutions:

- continuous monitoring of data
  - automatic alerts
  - scenario simulation (“what if sales increase 10%?”)
  - collaborative decision assistant
- 

# 13. Definition of Success (Important)

The product succeeds if:

A completely non-technical person can upload a spreadsheet and say:

“Now I understand my data and know what to do.”

---

# EVA-Datascience



## Autonomous Data Analysis Assistant

### Product Requirements Document

---

**From Raw Data to Real Decisions — Automatically.**

Version 1.0 | Confidential

# 1. Product Vision

EVA is a fully local, autonomous data science assistant that mirrors how a skilled data analyst thinks and works — from the moment a dataset is received, through understanding, cleaning, analysis, modeling, and finally to decisions and recommendations.

The core promise is simple:

**A person should be able to upload any dataset and walk away with a clear understanding of what it means, what will happen next, and what to do about it — without knowing a single line of code or a single statistical term.**

EVA does not function as a tool that requires the user to drive every decision. It functions as a reasoning analyst — one that thinks first, computes second, and always explains what it did and why.

---

## 2. The Core Design Insight

Most automated data tools assume the following workflow:

**Upload → Clean → Model → Output**

This is not how real data scientists work. A skilled analyst spends the first portion of their time not touching the data at all — they spend it asking questions about reality.

What system produced this data? What does each row actually represent? What decision is this dataset supposed to help make? What could go wrong if I misread it?

EVA is built around this insight. Every module in EVA — from cleaning to modeling to insight generation — is downstream of a reasoning step that happens before any computation begins.

EVA's workflow, therefore, mirrors a real analyst:

**Understand Reality → Clean → Explore → Engineer → Model  
→ Explain → Decide**

Each arrow in this chain passes structured context from one stage to the next. No stage is a black box. Every conclusion traces back to a reason.

---

## 3. Problem Statement

Working with data today has three compounding barriers that prevent non-technical users from deriving value on their own.

### 3.1 Time Is Wasted Before Intelligence Begins

The majority of a data analyst's time is consumed before any insight is produced. Data cleaning, formatting correction, handling missing values, removing duplicates, and resolving column inconsistencies account for 40 to 80 percent of total analysis time. Users are trapped doing maintenance work when they need answers.

### 3.2 The Knowledge Barrier

To answer even straightforward questions — why are sales falling, which patients are high risk, which customers will leave — a user must understand feature selection, statistical modeling, evaluation metrics, and interpretation. This eliminates students, business owners, healthcare analysts, and most non-technical professionals from accessing data intelligence on their own.

### 3.3 The Privacy and Trust Barrier

Users with sensitive data — medical records, client datasets, proprietary business information — are unwilling or legally unable to upload their data to external cloud platforms. They are left without options: either compromise on privacy or remain without insight.

---

## 4. Target Users

| User              | Primary Need                                                                           | What EVA Does For Them                                                            |
|-------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Student           | Understand their dataset for a project without knowing what analysis to run            | Explains data in plain language, suggests analysis direction, teaches as it works |
| Freelance Analyst | Process messy client CSVs quickly without losing hours to cleaning                     | Autonomously cleans and prepares data, delivers findings ready to present         |
| Business Owner    | Understand what their customer or sales data is saying without hiring a data scientist | Translates data into plain-language decisions with specific recommended actions   |

|                       |                                                                                     |                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Healthcare Researcher | Analyze patient data without it leaving a secure environment                        | Runs entirely on-device, HIPAA-compatible by design, explains clinical patterns |
| Non-Technical Founder | Make data-driven decisions without depending on a technical team for every question | Surfaces what matters, explains why, and recommends what to do next             |

## 5. The Analyst Workflow — EVA's Operating Sequence

Every stage EVA performs maps directly to what a skilled human data scientist would do. The stages are ordered deliberately. No stage begins until the previous one has produced its output. Each stage passes structured context to the next.

0

### Dataset Cognition

Understanding what the data represents before touching it

Before any processing begins, EVA reads the dataset and constructs a real-world understanding of it. This is the most critical stage. It determines everything that follows.

A real analyst receiving a new dataset does not immediately clean it or run a model. They first ask: What is this? What system generated these rows? What decision is this meant to inform? Who will act on the answer?

EVA performs this reasoning automatically. It inspects the raw structure of the dataset — column names, data types, value distributions, unique counts, null ratios, sample records — and from these signals infers what the dataset actually represents.

#### What EVA determines at this stage:

- What each row represents — a transaction, a patient visit, a sensor reading, a daily summary, an employee record
- The time behavior of the data — is it static, a time series, an event log, or panel data tracking the same entity across time
- The likely analytical goal — prediction, diagnosis, monitoring, segmentation, anomaly detection, or factor analysis
- The probable stakeholder — who will use the output and what decision they need to make
- Risks of misinterpretation — which columns are identifiers and must not be used as features, which columns may contain post-outcome information

EVA produces a structured Dataset Narrative that is passed to every subsequent stage. This narrative is the foundation of all downstream reasoning. Cleaning decisions reference it. Feature engineering uses it. Model selection is guided by it.

EVA will not proceed to any further stage without having completed this understanding step. This is a deliberate product rule: intelligence before computation, always.

## 1

### **Autonomous Data Cleaning**

Fixing the data based on what it actually represents

Once EVA understands the dataset, it cleans it — but not generically. Cleaning decisions are informed by the Dataset Narrative from Stage 0. A date column in a time series dataset is treated differently from a date column in a static snapshot. A numeric field in a medical dataset is handled with different assumptions than the same field in an e-commerce log.

#### **What EVA does at this stage:**

- Handles missing values using contextually appropriate strategies — mean imputation, forward-fill for time series, domain-specific defaults, or flagging where imputation would distort results
- Resolves inconsistent categorical entries — normalizing variations in spelling, case, and encoding into clean unified values
- Removes exact duplicates and flags near-duplicates for review
- Corrects data type mismatches — columns stored as text that should be numeric or datetime
- Detects and flags suspicious values — entries that fall outside statistically or domain-plausible ranges
- Identifies and preserves identifier columns as non-analytical fields

EVA does not ask the user to make manual decisions about cleaning. It handles all of this autonomously and produces a plain-language explanation of every change it made and why. The user can review what happened; they do not need to drive it.

## 2

### **Exploratory Analysis & Pattern Recognition**

Finding what is actually happening in the data

With clean data and a clear understanding of what it represents, EVA moves into structured exploration. This stage does not produce charts for the sake of charts. Every output connects back to the analytical goal identified in Stage 0.

#### **What EVA does at this stage:**

- Calculates distributions and identifies skew, outliers, and concentration in key variables



- Identifies correlations between variables — including which relationships are likely meaningful versus coincidental
- Detects natural groupings or segments within the data
- Surfaces anomalies and unusual patterns that warrant attention
- Identifies temporal trends for time-dependent datasets — seasonality, cycles, drift, step changes

All outputs from this stage are expressed in plain English. EVA does not present a matrix of correlation coefficients and leave the user to interpret it. It explains: "Age and monthly purchase frequency are strongly linked to whether a customer stays or leaves. Customers who haven't purchased in 90 days are significantly more likely to churn."

This is the stage where EVA begins teaching — not just analyzing. Users come away understanding not just what the data shows, but why certain patterns matter.

3

Domain-Aware Feature Engineering

Preparing the data the way an expert in that field would

Raw columns are rarely the best representation of what matters. A skilled analyst who works in healthcare builds different derived variables than one who works in finance or e-commerce. EVA performs this same domain-aware transformation automatically.

Using the domain and analytical goal established in Stage 0, EVA identifies what derived features would be meaningful, why they matter in this specific context, and how they should be constructed.

Examples of how domain changes the engineering:

| Domain        | Raw Columns                 | Derived Feature          | Why It Matters                            |
|---------------|-----------------------------|--------------------------|-------------------------------------------|
| Healthcare    | age, height, weight         | BMI category             | Captures nonlinear health risk thresholds |
| E-commerce    | order_date, last_order_date | Days since last purchase | Core signal for churn likelihood          |
| Finance       | daily_close_price           | 7-day rolling average    | Smooths noise, reveals trend direction    |
| HR            | hire_date, today            | Tenure in months         | Tenure correlates with attrition risk     |
| Manufacturing | sensor_readings over time   | Rolling variance         | Early indicator of equipment failure      |

EVA explains every feature it proposes in two ways: the statistical reason it was created, and the real-world business meaning it carries. This is not optional — every feature suggestion includes both.

EVA also identifies what it will not do: it will not use identifier columns as features, it will not create features that encode information only available after the outcome has occurred, and it will not produce features that have no interpretable meaning in the domain.

## 4

### Model Reasoning & Selection

Choosing the right approach, not just the default one

EVA selects a modeling approach based on the problem type determined in Stage 0 and the features engineered in Stage 3. It does not default to a single algorithm. It reasons through the choice.

#### How EVA approaches model selection:

- Identifies the problem type — binary classification, multi-class classification, regression, time series forecasting, clustering, or anomaly detection
- Selects candidate approaches appropriate to the data size, feature types, and interpretability requirements of the stakeholder
- Explains why each candidate approach is being considered and what its tradeoffs are
- Trains and evaluates multiple approaches where appropriate
- Selects the final model based on performance and interpretability, not complexity

For users who need to explain their results — to a manager, a professor, a client, a medical team — EVA prioritizes interpretable models and supplements them with explanation tools. A model that cannot be explained is not useful to most of EVA's users.

EVA will not present model results before it has presented what the dataset represents. This rule is enforced at every stage.

## 5

### Insight Generation & Explanation

Translating results into language anyone can understand

Model output in raw form — accuracy scores, feature importance arrays, coefficient tables — is not useful to EVA's users. EVA translates every result into plain language that connects back to the real-world context established in Stage 0.

#### What EVA produces at this stage:

- A narrative explanation of what the model found — not statistical jargon, but a story about the data

- The top factors that drive the outcome, explained in terms the stakeholder understands
- Confidence levels for predictions, expressed in plain language
- Specific flagged cases — which customers, which patients, which products — are high priority and why
- A clear explanation of model limitations and where the results should be trusted with caution

EVA is designed to teach as it analyzes. A student using EVA to analyze a dataset for a class project should come away not just with results, but with a better understanding of why those results make sense.

## 6

### Decision Guidance

Telling the user what to actually do next

Analysis is only valuable when it leads to action. EVA does not stop at insight — it translates insight into specific, actionable recommendations.

This is the stage where EVA acts as a business consultant, not just a statistician. Given what the data shows, given what the model predicts, given who the stakeholder is — what should happen next?

#### Examples of decision guidance EVA produces:

- For a business owner: "These 47 customers show strong churn indicators. Prioritize them for a retention outreach in the next 14 days. Customers in this group who have been inactive for more than 60 days have the highest urgency."
- For a healthcare researcher: "These patient profiles match the risk factors associated with readmission. Flag them for follow-up scheduling before discharge."
- For a student: "Your strongest predictor is study hours per week. Increasing it from 5 to 10 hours is associated with an average grade improvement of one letter grade in this dataset."

EVA always distinguishes between what the data shows and what it recommends as action. It is transparent about the difference between correlation and causation, and it makes clear when a recommendation is a strong signal versus a weaker probabilistic suggestion.

## 7

### Conversational Refinement

The user asks; EVA answers with context

After the full analysis is complete, the user can ask follow-up questions in plain English. EVA retains the full context of the session — the dataset narrative, the cleaning decisions, the

features created, the model results, and the insights generated — and uses all of it to answer questions precisely.

**Examples of what users ask at this stage:**

- "Why did sales drop in Q3?" — EVA traces the answer through the data, not just the model
- "What would happen if we discount this product category?" — EVA reasons from the patterns it found
- "Explain this to me like I'm not a data person" — EVA rewrites its explanation in simpler terms
- "Which variable matters most for predicting churn?" — EVA answers with both the statistical answer and the business meaning
- "Can I use this for my report?" — EVA produces a formatted summary ready for presentation

Every answer traces back to something EVA actually found in the data. EVA does not generate plausible-sounding responses that are disconnected from the dataset. It grounds every answer in evidence it can point to.

---

## 6. Multi-Agent Reasoning Architecture

EVA's analytical pipeline is powered by a council of specialized AI agents, not a single model. This architecture is motivated by a simple principle: single models hallucinate. A council of agents that reason independently and reach consensus produces more reliable results.

Each agent in the council has a defined role, a defined scope, and a defined output format. No agent operates outside its domain. The Council Chairman synthesizes their outputs and resolves disagreements before any result is passed to the user.

| Agent                | Role                                                                                    | Scope                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| The Analyzer         | The Statistician — performs deep structural and distributional analysis of the data     | Distribution analysis, correlation detection, pattern identification, outlier detection |
| The Cleaner          | The Janitor — generates and executes cleaning operations with justification             | Missing value handling, type correction, deduplication, formatting normalization        |
| The Feature Engineer | The Creative — proposes domain-appropriate derived features with formulas and rationale | Feature derivation, transformation suggestions, domain-specific variable creation       |
| The Model Architect  | The Engineer — selects algorithms, configures training, evaluates results               | Model selection, hyperparameter reasoning, evaluation interpretation                    |

|              |                                                                                                                   |                                                                        |
|--------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| The Chairman | The Decision-Maker — synthesizes confidence scores from all agents, resolves conflicts, produces the final output | Consensus synthesis, conflict resolution, quality gating before output |
|--------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|

Agents analyze independently to avoid confirmation bias. Where agents disagree — for example, on whether to impute or drop a missing-value column — the Chairman receives both positions with confidence scores and resolves the conflict. The resolution and the reasoning behind it are visible to the user.

---

## 7. Functional Requirements

### 7.1 Dataset Cognition (Stage 0)

- EVA must identify what each row in a dataset represents
- EVA must classify the time behavior of the dataset — static, time series, event log, or panel
- EVA must infer the likely analytical goal from structure and content alone
- EVA must detect identifier columns and prevent them from being used as model features
- EVA must produce a Dataset Narrative in plain English before any further processing begins
- EVA must not proceed to cleaning until Stage 0 is complete

### 7.2 Data Cleaning (Stage 1)

- EVA must handle missing values autonomously using contextually appropriate strategies
- EVA must resolve categorical inconsistencies without manual user input
- EVA must detect and remove exact duplicates
- EVA must correct data type mismatches
- EVA must flag suspicious values with explanations
- EVA must explain every cleaning decision in plain language

### 7.3 Exploratory Analysis (Stage 2)

- EVA must identify the most important variables relative to the analytical goal
- EVA must surface correlations and explain their practical significance
- EVA must detect anomalies and unusual patterns
- EVA must identify temporal trends for time-dependent datasets
- All exploratory outputs must be expressed in plain English, not statistical notation

## 7.4 Feature Engineering (Stage 3)

- EVA must infer the dataset domain and retrieve domain-specific feature engineering knowledge
- EVA must propose derived features with formulas, statistical justification, and business meaning
- EVA must not propose features that use identifier columns
- EVA must not propose features that encode post-outcome information
- Every proposed feature must include its expected impact on model performance

## 7.5 Modeling (Stage 4)

- EVA must select modeling approaches based on problem type, not defaults
- EVA must explain why each approach was selected or rejected
- EVA must evaluate multiple candidate approaches where appropriate
- EVA must prioritize interpretable models for stakeholders who need to explain their results
- EVA must not present model results before presenting what the dataset represents

## 7.6 Insight Generation (Stage 5)

- EVA must translate all model outputs into plain-language explanations
- Every conclusion must include a reason, a confidence level, and an interpretation
- EVA must identify and flag specific high-priority cases — not just aggregate patterns
- EVA must clearly state limitations and where results should be treated with caution

## 7.7 Decision Guidance (Stage 6)

- EVA must produce specific, actionable recommendations — not general observations
- EVA must tailor recommendations to the identified stakeholder type
- EVA must distinguish between correlation-based observations and causal claims
- EVA must indicate the confidence level behind each recommendation

## 7.8 Conversational Refinement (Stage 7)

- EVA must retain full session context across the conversation
- EVA must answer follow-up questions grounded in the actual data it analyzed
- EVA must be able to explain any conclusion in simpler language on request

- EVA must be able to produce a formatted summary suitable for presentation or reporting
- 

## 8. Non-Functional Requirements

### 8.1 Privacy — Local First

EVA is designed to run entirely on the user's own machine. No data, no query, no intermediate result is transmitted to any external server. This is not a feature; it is a design constraint that governs every architectural decision.

- All AI inference runs locally
- All data processing runs locally
- No external API calls for any core functionality
- Compatible with air-gapped environments

### 8.2 Transparency

EVA must be able to account for every decision it makes. Users can always ask why EVA did something, and EVA must provide a traceable, honest explanation.

- Every cleaning decision is logged and explained
- Every feature suggestion includes its rationale
- Every model choice includes the reasoning behind it
- Confidence levels are always surfaced, not hidden

### 8.3 Usability

The benchmark for usability is a 10th-grade student with no data science background. If such a user cannot complete an analysis session and understand what they found, EVA has not met its usability standard.

- No technical prerequisites required
- No configuration required to begin an analysis
- All outputs are expressed in plain language by default
- Technical detail is available on request but never imposed

### 8.4 Speed

EVA should feel responsive and interactive, not like a batch process. The user should not be waiting passively for long periods.

- Initial dataset cognition and structural analysis: under 30 seconds for typical datasets
- Feature engineering suggestions: under 15 seconds
- Model training and evaluation: proportional to dataset size, with progress indication
- Conversational responses: under 10 seconds

---

## 9. Definition of Success

EVA succeeds when a completely non-technical person can upload a spreadsheet and say, with genuine understanding:

|    **"Now I understand my data and I know what to do."**

This is not a metric. It is the standard against which every product decision should be evaluated.

### 9.1 Operational Metrics

| Metric                                                      | Target          |
|-------------------------------------------------------------|-----------------|
| Users who complete a full analysis on first attempt         | Over 80%        |
| Time from upload to first meaningful insight                | Under 5 minutes |
| Users who report understanding their data after session     | Over 85%        |
| Users who can present results without additional processing | Over 70%        |
| Feature suggestions rated as relevant by domain experts     | Over 75%        |
| Hallucination rate in feature or insight generation         | Under 5%        |

---

## 10. Future Directions

The following capabilities are explicitly out of scope for the current milestone but represent the natural evolution of EVA as the core pipeline matures.



- Continuous data monitoring — EVA watches a connected dataset over time and surfaces changes automatically
  - Scenario simulation — users ask "what if" questions and EVA reasons through the implications
  - Automated feature validation — EVA tests proposed features against model improvement before recommending them
  - SQL and database integration — EVA connects directly to structured data sources rather than requiring CSV upload
  - Multi-dataset reasoning — EVA understands how two related datasets connect and what can be learned from their relationship
  - Collaborative sessions — multiple users can work with the same EVA analysis simultaneously
  - Deep learning support — for image, audio, and unstructured text datasets
- 

## 11. Product Rules

These are non-negotiable constraints on EVA's behavior. They exist to protect the quality and integrity of every user interaction.

1. EVA will never present model results before explaining what the dataset represents.
  2. EVA will never use identifier columns as model features, regardless of what other systems might do.
  3. EVA will never produce a conclusion without a reason that traces back to the data.
  4. EVA will never pretend to be more confident than it is — every output includes an honest confidence signal.
  5. EVA will never transmit user data outside the user's device.
  6. EVA will never require the user to make a technical decision to proceed — all technical choices are EVA's responsibility.
  7. EVA will always explain what it did, in plain language, after doing it.
- 

**EVA — Stop Cleaning. Start Understanding.**

# GAL-Global Analysis Ledger



# EVA Core System Document

## The Global Analysis Ledger (GAL)

### 1. What the Global Analysis Ledger Is

The **Global Analysis Ledger (GAL)** is the permanent shared memory of a dataset analysis session.

Every EVA agent:

- reads from it before acting
- writes to it after acting
- justifies its decisions inside it

It is not a cache.

It is not a log file.

It is the **official reasoning record** of the analysis.

You can think of it as:

The notebook a human data scientist keeps while working — except structured and enforced.

No stage is allowed to run without consulting the GAL.

---

### 2. Why the GAL Is Necessary

Most AI pipelines fail because each step works in isolation.

Example failure:

- Cleaning removes a column
- Feature engineering needed it

- Model becomes biased
- Nobody knows why

The problem isn't the model.

The problem is **lost reasoning context**.

EVA solves this by enforcing a single shared source of truth.

The GAL guarantees:

- continuity between agents
- explainability
- reproducibility
- debuggability
- trust

Every output EVA gives to the user must be traceable to entries inside the GAL.

---

### 3. Lifecycle of a Session

When a dataset is uploaded:

1. A new **Analysis Session** is created
2. A blank GAL is initialized
3. All subsequent agents operate only within this session

The session ends only after:

- report generation
- or user termination

No step overwrites previous reasoning.

It can only append or annotate.

---

### 4. Structure of the Global Analysis Ledger

The GAL is divided into sections.  
Each section corresponds to how a real analyst thinks.

---

## SECTION 1 — Dataset Identity

Created by: Dataset Profiler

Purpose: establish what the data *is*

Contains:

- dataset description
- row meaning (transaction, patient, user, sensor reading, etc.)
- time behavior (static / time series / event log / panel)
- suspected domain
- column roles:
  - identifier
  - feature
  - target candidate
  - timestamp
    - dataset size
    - data health summary

This section becomes the **reference reality**.  
Every later decision must be consistent with it.

No agent may contradict it without explicitly updating it.

---

## SECTION 2 — User Intent

Created by: Question Builder

Purpose: establish what the user wants to decide.

Contains:

- user goal (prediction / explanation / monitoring / segmentation)
- stakeholder type (student, business owner, researcher, analyst)
- success definition
- constraints:
  - interpretability importance
  - speed vs accuracy
  - actionable vs exploratory

Why this matters:

Cleaning, feature engineering, and visualizations depend heavily on intent.  
The same dataset should be treated differently depending on the decision goal.

---

## SECTION 3 — Data Integrity Record

Created by: Data Repair & Integrity Layer

Purpose: document all modifications to the data.

For each operation the ledger records:

- problem detected
- why it matters
- chosen strategy
- alternatives considered
- effect on dataset

Examples recorded:

- missing value treatment
- duplicate removal
- outlier handling
- type correction
- imbalance handling

Important rule:

EVA is never allowed to silently modify the dataset.

Every change must be justified in the GAL.

---

## SECTION 4 — Exploratory Findings

Created by: Exploration / Pattern Recognition Agent

Purpose: record what patterns actually exist in the data.

Contains:

- distributions
- correlations
- trends
- clusters
- anomalies
- temporal behaviors

This section contains **observations only**.

No explanations yet.

Example:

“Customers inactive for 60+ days show significantly higher churn rate.”

No reasoning. Only evidence.

---



## SECTION 5 — Investigation & Hypothesis Engine (NEW — Critical)

Created by: Investigation & Hypothesis Engine

This is the missing stage in most AutoML systems.

After EDA, EVA must ask:

“Why might these patterns exist in the real world?”

The system generates **candidate explanations**.

This converts:

data patterns → real-world reasoning

---

## What the Engine Does

For each important observation, EVA proposes multiple hypotheses.

Example:

Observation:

Sales dropped sharply in Q3.

Hypotheses:

1. seasonal demand shift
2. price increase effect
3. product availability issue
4. customer retention failure

Another:

Observation:

High BMI correlates with hospital readmission.

Hypotheses:

1. comorbidity risk
  2. mobility limitations
  3. medication adherence difficulty
- 

## Structure of a Hypothesis Entry



Each hypothesis stored in GAL must include:

- related observation
- explanation
- supporting evidence from data
- conflicting evidence (if any)
- plausibility score (low / medium / high)

Important rule:

EVA never treats correlation as explanation until evaluated here.

This stage is essential because:

Without it → system is just pattern detection

With it → system becomes reasoning

---

## SECTION 6 — Feature Reasoning

Created by: Feature Intelligence Engine

Purpose: explain why new features are created.

For every engineered feature:

- what raw columns were used
- real-world meaning
- statistical purpose
- expected impact

This ties modeling preparation to domain logic.

---

## SECTION 7 — Visualization Plan

Created by: Visualization Planner

Contains:

- questions visuals should answer
- what comparisons matter

- what trends need illustration
- target audience

This is not charts.

This is **why charts exist**.

---

## SECTION 8 — Evidence Register

This is extremely important.

Every conclusion EVA ever gives must cite:

- which finding
- which hypothesis
- which feature
- which statistic

Think of it as citations in a research paper.

If EVA cannot cite the ledger → it cannot claim it.

---

## SECTION 9 — Recommendations & Decisions

Created by: Decision Guidance Module

Contains:

- suggested actions
- affected groups
- urgency level
- expected impact
- confidence level

Important rule:

Recommendations must reference hypotheses + evidence.

No free opinions.

---

## SECTION 10 — Report Memory

Created by: Report Generator

Stores:

- final narrative
- summary explanations
- presentation-ready insights

Allows user to revisit analysis later.

---

## Operational Rules (Very Important)

1. No agent may skip reading the GAL
  2. No agent may silently alter data
  3. Every claim must cite ledger evidence
  4. Hypotheses must precede recommendations
  5. Reports must be reproducible from GAL alone
- 

## Why This Changes EVA Completely

Typical AutoML:

dataset → model → metrics

EVA with GAL:

dataset → understanding → evidence → hypotheses → decisions

This is the difference between:

**a tool** and **an analyst**.

# IHE-Investigation & Hypothesis Engine



# EVA System Module Specification

## Investigation & Hypothesis Engine (IHE)

---

### 1. Purpose

The **Investigation & Hypothesis Engine (IHE)** is responsible for transforming observed data patterns into possible real-world explanations.

Exploratory analysis identifies *what is happening*.

The Investigation & Hypothesis Engine attempts to reason about *why it might be happening*.

This module converts statistical observations into structured, testable hypotheses. It ensures EVA behaves like an analyst rather than a pattern-reporting tool.

Without this module, the system produces correlations.

With this module, the system produces reasoning.

The IHE operates immediately after the Exploratory Findings stage and before Feature Reasoning, Decision Guidance, and Modeling.

---

### 2. Design Principle

A critical rule governs EVA:

**Correlation is not explanation.**

Exploratory analysis can detect relationships between variables, but a relationship does not automatically imply a cause. The IHE exists to interpret those relationships carefully and responsibly.

The module never claims a hypothesis is true.

It only proposes plausible explanations supported by available evidence.

The output of this module feeds the Global Analysis Ledger (GAL) and becomes the reasoning backbone for all downstream modules.

---

### 3. Inputs

The IHE does not read raw data directly.

Instead, it consumes structured outputs written to the Global Analysis Ledger.

It reads:

- Observed correlations
- Distribution abnormalities
- Trends and temporal patterns
- Detected anomalies
- Cluster/segment characteristics
- Dataset identity and domain
- User intent

The module therefore reasons from interpreted observations, not raw numbers.

---

### 4. Core Responsibilities

The Investigation & Hypothesis Engine must:

1. Identify important observations from exploratory findings
2. Determine which observations require explanation
3. Generate multiple plausible explanations
4. Connect explanations to real-world mechanisms
5. Evaluate supporting and conflicting evidence
6. Assign plausibility levels
7. Record results into the Global Analysis Ledger

The engine must generate more than one explanation whenever possible.

Single-explanation outputs are forbidden unless evidence is overwhelming.

---

### 5. What Counts as an “Observation”

The module only investigates significant findings. Examples include:

- Strong correlation between variables
- Sudden trend changes
- Highly skewed distributions
- Segment differences
- Predictive variables emerging
- Outliers with real-world importance
- Time-dependent shifts
- Behavior differences between groups

Minor statistical noise must not trigger hypothesis generation.

---

## 6. Hypothesis Generation Process

For each selected observation, the engine follows five reasoning steps:

### Step 1 — Interpret the Observation

Translate statistical output into real-world meaning.

Example:

“Customers inactive for 60 days have higher churn rate”  
becomes

“Customer inactivity is associated with leaving the service.”

---

### Step 2 — Consult Dataset Context

Use dataset identity and domain knowledge from the Global Analysis Ledger to understand what the observation could represent in reality.

The same pattern has different meanings in different domains.

Example:

- inactivity in healthcare → missed follow-ups
  - inactivity in e-commerce → disengagement
- 

### Step 3 — Generate Candidate Explanations

Produce multiple possible causes.

Each hypothesis must be a real-world mechanism, not a mathematical statement.

Examples:

Observation: sales decreased in one quarter

Possible hypotheses:

- seasonal demand shift
  - product supply issue
  - price change effect
  - competitor entry
  - customer retention drop
- 

#### **Step 4 — Evidence Matching**

Evaluate whether the dataset supports or contradicts each explanation.

The engine must actively search for:

- supporting indicators
- missing indicators
- contradictory signals

A hypothesis may remain plausible even without strong support, but conflicting evidence must reduce confidence.

---

#### **Step 5 — Assign Plausibility Level**

Each hypothesis is assigned one of three levels:

High Plausibility — strong evidence supports it  
Moderate Plausibility — consistent but weak evidence  
Low Plausibility — limited or conflicting evidence

No hypothesis is labeled “true”.

---

### **7. Output Format (Written to Global Analysis Ledger)**

Each hypothesis entry must include:



Observation:

The pattern being explained.

Hypothesis:

The proposed real-world explanation.

Supporting Evidence:

What in the dataset supports it.

Contradictory Evidence:

What weakens it, if anything.

Plausibility Level:

Low / Moderate / High

Confidence Note:

Limitations or uncertainties.

Example:

Observation:

Customers with long inactivity periods churn more often.

Hypothesis:

Customer disengagement leads to churn.

Supporting Evidence:

Churned customers show reduced activity before leaving.

Contradictory Evidence:

Some highly active users also churn.

Plausibility Level:

High

Confidence Note:

Behavioral patterns suggest risk but not guaranteed causation.

---

## 8. Rules & Restrictions

The Investigation & Hypothesis Engine must obey the following:

1. It must not invent data not present in the dataset.

2. It must not claim causation as fact.
  3. It must generate at least two hypotheses when possible.
  4. It must reference observations from the Global Analysis Ledger.
  5. It must record uncertainties explicitly.
  6. It must avoid domain-irrelevant explanations.
- 

## 9. Relationship With Other Modules

The outputs of the IHE are used by:

Feature Engineering

→ to design meaningful derived variables

Modeling

→ to choose appropriate prediction targets

Visualization

→ to design explanatory plots

Decision Guidance

→ to justify recommendations

Report Generator

→ to explain findings to users

If the IHE is skipped, downstream reasoning becomes unsupported and recommendations lose credibility.

---

## 10. Why This Module Is Critical

Traditional data pipelines:

Data → Model → Metrics

EVA pipeline:

Data → Observations → Hypotheses → Evidence → Decision

The Investigation & Hypothesis Engine is the step that converts analysis into understanding.

It enables EVA to answer not only:

“What is happening?”

but also:

“What might be causing it?”

and therefore:

“What should we do?”

# Dataset Profiler & Semantic Understanding



# EVA System Module Specification

## Dataset Profiler & Semantic Understanding (DPSU)

---

### 1. Purpose

The **Dataset Profiler & Semantic Understanding (DPSU)** module is responsible for building EVA's first mental model of an uploaded dataset.

Before any cleaning, questioning, feature engineering, visualization, or modeling can begin, EVA must determine what the dataset represents in the real world.

This module answers the foundational question:

**“What does each row actually mean?”**

The DPSU does not analyze patterns and does not make predictions.  
Its only responsibility is understanding structure and meaning.

The outputs of this module populate the *Dataset Identity* section of the Global Analysis Ledger (GAL) and become the reference context for the entire session.

No downstream module is allowed to operate without this stage being completed.

---

### 2. Design Philosophy

Most automated data tools treat datasets as tables.

EVA treats datasets as **records of real-world events**.

A dataset is not merely columns and rows.

It is evidence of a process:

- purchases
- patient visits
- sensor readings
- employee records
- transactions
- measurements over time

If EVA misidentifies what a row represents, every later step becomes incorrect.

Therefore, DPSU prioritizes interpretation over computation.

---

### **3. Inputs**

The module receives:

- the raw dataset
- column names
- sample records
- data types
- missingness patterns
- value distributions

It does not require user input.

---

### **4. Core Responsibilities**

The Dataset Profiler & Semantic Understanding module must:

1. Inspect dataset structure
  2. Infer column roles
  3. Detect dataset behavior over time
  4. Identify potential target variables
  5. Determine dataset domain
  6. Detect risks of misinterpretation
  7. Record findings into the Global Analysis Ledger
- 

### **5. Column Role Identification**

Each column must be assigned a role.

This classification becomes permanent context for all later modules.

Possible roles:

Identifier

Uniquely identifies an entity (e.g., customer ID, patient ID, order number)

Feature

Describes attributes used for analysis

Target Candidate

Potential outcome variable

Timestamp

Represents time information

Categorical Attribute

Represents categories or labels

Numeric Measurement

Represents quantities

Derived/Redundant Field

Represents repeated or encoded information

The module must also detect columns that **must never be used as predictive features**, especially identifiers and post-outcome data.

---

## 6. Row Meaning Inference

The system must determine what one row represents.

Examples:

- one purchase transaction
- one patient visit
- one sensor reading
- one daily summary
- one employee record
- one customer profile

This is one of the most critical outputs because it determines:

- feature engineering strategy
  - visualization strategy
  - modeling approach
  - decision interpretation
- 

## 7. Time Behavior Classification

The module must classify the dataset into one of the following:

Static Snapshot

Each row independent of time

Time Series

Measurements recorded across time

Event Log

Events occurring at irregular intervals

Panel Data

Same entities tracked across time

Many later decisions — including cleaning and hypothesis generation — depend on this classification.

---

## 8. Target Variable Detection

The module must identify possible outcomes the dataset may be used to predict or explain.

Signals for a target candidate include:

- binary status columns
- outcome labels
- final measurements
- completion indicators
- change indicators

If multiple candidates exist, all are recorded.

The module does not finalize the target — the Question Builder later confirms intent.

---



## 9. Domain Inference

The module must infer the real-world domain of the dataset.

Possible domains include:

- healthcare
- finance
- e-commerce
- marketing
- education
- HR
- manufacturing
- sensors/IoT

Domain inference is required because:

Feature engineering depends on domain knowledge

Hypotheses depend on real-world context

Visualization relevance depends on stakeholder needs

---

## 10. Data Health Assessment

The module performs a non-destructive inspection of data quality and records warnings.

It must detect:

- missing value concentration
- duplicated records
- suspicious ranges
- inconsistent formatting
- sparse columns
- extreme imbalance
- potential leakage fields

Important:

This module does not fix problems.

It only reports them.

The Data Repair module later acts on this information.

---

## 11. Misinterpretation Risk Detection

The module must identify columns that may cause incorrect analysis.

Examples:

- ID columns accidentally used as predictors
- future information available before outcome
- encoded target leakage
- derived labels

These risks are written explicitly into the Global Analysis Ledger so later modules cannot misuse them.

---

## 12. Output Written to the Global Analysis Ledger

The module writes the **Dataset Identity Record** containing:

Dataset Summary

What the dataset appears to represent

Row Meaning

Real-world interpretation of each row

Column Role Map

Classification of every column

Time Behavior

Static / Time series / Event / Panel

Target Candidates

Possible outcomes

Domain Guess

Likely domain

Data Health Warnings

Quality observations

Misinterpretation Risks

Fields to avoid or treat carefully

---

### 13. Operational Rules

1. No cleaning may occur before this module finishes.
  2. No features may be engineered without column role classification.
  3. No hypotheses may be generated without row meaning.
  4. The Question Builder must consult this module before asking users anything.
- 

### 14. Why This Module Is Critical

Typical systems assume:  
columns → features

EVA determines:  
records → reality

This module ensures that every later decision is grounded in what the dataset actually represents.

If this stage is wrong, the entire analysis is wrong.

The Dataset Profiler & Semantic Understanding module is therefore the **foundation of the EVA reasoning pipeline**.

# Question Builder & Intent Inference



# EVA System Module Specification

## Question Builder & Intent Inference (QBII)

---

### 1. Purpose

The **Question Builder & Intent Inference (QBII)** module determines the user's analytical objective before any data modification or interpretation occurs.

After the Dataset Profiler understands the structure of the dataset, EVA must understand the *decision context*.

A dataset does not have a single correct analysis.  
The correct analysis depends on the user's goal.

The QBII module identifies:

- what the user is trying to learn
- what decision they want to make
- what type of answer is useful to them

The module writes this information into the **User Intent section** of the Global Analysis Ledger (GAL).

All later modules must adapt their behavior based on this.

---

### 2. Design Philosophy

Traditional tools ask users to configure settings manually.  
EVA instead conducts a guided conversation.

The system does not ask technical questions such as:

- choose target variable
- choose algorithm
- choose evaluation metric

Instead, EVA asks real-world questions a human analyst would ask a client.

The module translates human language into structured analytical intent.

---

### 3. When It Runs

QBII runs **after** Dataset Profiler & Semantic Understanding completes.

It must not run earlier because:

- questions depend on dataset type
- irrelevant questions reduce trust

Example:

If the dataset has no outcome column, prediction-focused questions are avoided.

---

### 4. Inputs

The module reads from the Global Analysis Ledger:

- dataset domain
- row meaning
- time behavior
- target candidates
- data quality summary

It also receives:

- user responses to guided questions
- 

### 5. Core Responsibilities

The Question Builder & Intent Inference module must:

1. Generate context-aware questions
  2. Minimize user effort
  3. Infer analysis goal
  4. Identify stakeholder type
  5. Determine success criteria
  6. Identify constraints and priorities
  7. Write structured intent into the Global Analysis Ledger
-

## 6. Question Generation

The system dynamically constructs a short interview (3–5 questions). Questions must be simple and decision-focused.

The system must never ask technical configuration questions.

---

### Types of Questions

#### Goal Questions

Determine what the user wants to achieve.

Examples:

- Are you trying to predict an outcome or understand behavior?
  - Do you want to know why something happened or what will happen?
- 

#### Decision Questions

Identify the real-world action tied to the analysis.

Examples:

- What decision will this analysis help you make?
  - Who will use the result?
- 

#### Priority Questions

Understand tradeoffs.

Examples:

- Is explanation more important than accuracy?
  - Do you need simple results for a report or detailed insights?
- 

#### Time Context Questions

Only asked for time-dependent datasets.

Examples:

- Are you tracking changes over time?
  - Do you want to detect trends or forecast future values?
-

## 7. Intent Inference

Based on responses, the module must infer:

Primary Analysis Type:

- prediction
- explanation
- segmentation
- anomaly detection
- monitoring
- reporting

Stakeholder Type:

- student
- business owner
- researcher
- analyst
- manager

Decision Horizon:

- immediate action
- planning
- long-term understanding

Interpretability Requirement:

- high (needs explanation)
- moderate
- low (performance priority)

Success Definition:

What outcome will make the analysis useful.

---

## 8. Target Confirmation

If the Dataset Profiler identified potential target variables, QBII confirms with the user which outcome is relevant.

If none exists, QBII switches analysis type automatically to:

- explanation
- segmentation
- monitoring

The system must never force prediction when no valid outcome exists.



---

## 9. Output Written to the Global Analysis Ledger

The module writes the **User Intent Record** containing:

User Goal

Primary analysis objective

Decision Context

Real-world decision supported

Stakeholder Type

Who will use the results

Selected Target (if applicable)

Outcome of interest

Analysis Priority

Interpretability vs performance

Success Criteria

What defines a useful result

Constraints

Time, simplicity, or reporting needs

---

## 10. Operational Rules

1. The Data Repair module must consult this record before modifying data.
  2. Feature Engineering must adapt features to the chosen objective.
  3. Visualization must adapt complexity to stakeholder type.
  4. Report Generator must tailor language to audience.
  5. Modeling (future stage) must select methods consistent with interpretability requirements.
- 

## 11. Failure Handling

If the user gives minimal answers, the module must infer intent conservatively using dataset structure and domain.

If user responses conflict, the system prioritizes:  
decision clarity → interpretability → complexity

The system may summarize its understanding and ask the user to confirm.

---

## **12. Why This Module Is Critical**

Without intent, analysis is arbitrary.

Two users can upload the same dataset and need completely different outputs:

A student → explanation for a report

A business owner → action plan

A researcher → pattern discovery

QBII ensures EVA solves the correct problem before touching the data.

This module transforms EVA from a data processor into a collaborative analyst.

# Feature Engineering RAG Engine

# Project: EVA — Domain-Aware Feature Engineering RAG Engine

**Module Name:** Feature Intelligence Engine (FIE)

**Type:** Multi-Domain Retrieval-Augmented Generation System

**Scope:** Feature Engineering Only (NOT modeling, NOT visualization, NOT chat)

---

## 1. Purpose

The Feature Intelligence Engine (FIE) is responsible for automatically generating **domain-appropriate feature engineering suggestions** from an uploaded dataset.

The system must:

- identify the real-world domain of a dataset
- retrieve domain expertise
- propose useful derived features
- justify them statistically AND in business terms

Goal:

Turn a raw CSV into a “data scientist-prepared dataset plan” without human intervention.

This replaces the part where a real data scientist thinks:

“What variables should I create before training the model?”

---

## 2. Problem Statement

Current AutoML tools:

- clean data
- train models
- tune hyperparameters

But they **do not understand context**.

Example:

- Healthcare dataset → needs risk stratification features
- Marketing dataset → needs funnel & cohort features
- Finance dataset → needs temporal & volatility features

Without domain-aware feature engineering:

- models overfit
- models underperform
- insights are meaningless

EVA's Feature Intelligence Engine solves:

Lack of domain-aware feature engineering in automated ML pipelines.

---

### 3. Objectives

#### Functional Objectives

1. Automatically infer dataset domain
2. Retrieve domain feature engineering knowledge
3. Suggest new derived features
4. Provide formulas for feature creation
5. Provide business meaning of each feature
6. Provide expected ML impact

#### Non-Functional Objectives

- Fully local (no external APIs)
  - Runs on laptop
  - Response < 15 seconds
  - Deterministic structure of output
  - Reproducible feature plan
- 

### 4. User Persona

| User    | Need                           |
|---------|--------------------------------|
| Student | learn how features are created |

|                    |                       |
|--------------------|-----------------------|
| Analyst            | faster preprocessing  |
| Startup founder    | actionable insights   |
| Non-technical user | understand their data |

---

## 5. System Inputs

Input Source: CSV Upload

The engine does NOT receive natural language questions.

### Input Data Provided to RAG

The system must automatically generate a dataset profile:

#### Dataset Profile Object

- row count
  - column names
  - data types
  - missing values
  - basic statistics
  - sample records
- 

## 6. System Outputs

The RAG module must output a structured **Feature Plan JSON**:

```
{
  "domain": "healthcare",
  "problem_type": "binary_classification",
  "features": [
    {
      "name": "bmi_category",
      "formula": "BMI grouped into Underweight/Normal/Overweight/Obese",
      "type": "categorical",
      "why_it_matters": "captures nonlinear health risk patterns",
      "business_meaning": "patient risk stratification",
      "expected_ml_impact": "improves classification boundary"
    }
  ]
}
```

```
}  
]  
}
```

Important:

This is NOT optional — the output format must be strictly structured.

---

## 7. Functional Components

### 7.1 Dataset Profiler

Purpose: Convert CSV → semantic description

Responsibilities:

- detect column types
- detect missingness
- calculate ranges
- detect target variable candidates

Output:

Dataset summary text

---

### 7.2 Domain Classifier

Purpose: Identify real-world system represented by the dataset

Possible domains:

- Healthcare
- Finance
- Ecommerce
- Marketing
- HR
- Manufacturing

Method:

LLM classification using dataset summary

Output:

`domain_label`

---

### 7.3 RAG Router

Purpose: Select correct domain knowledge base

Behavior:

- load correct vector DB
  - retrieve top-k domain documents
- 

### 7.4 Domain Knowledge Bases

Each domain has its own vector database.

Each DB contains:

- feature engineering patterns
- domain metrics
- derived variable logic
- transformation rules

Example documents:

- “RFM analysis for ecommerce”
  - “Lag features in time series”
  - “Clinical risk scoring features”
- 

### 7.5 Retrieval Module

Responsibilities:

- embed dataset summary
  - retrieve relevant domain docs
  - pass context to LLM
- 

### 7.6 Feature Engineering Agent



Purpose:

Generate feature suggestions using retrieved knowledge.

The LLM must:

- read schema
  - read domain docs
  - reason
  - produce feature plan
- 

## 8. Prompt Design Requirements

The system prompt MUST include:

1. Domain role
2. Feature engineering focus
3. Structured output requirement

Required behavior:

- no hallucinated columns
- only use existing columns
- no generic advice

Forbidden behavior:

- model training suggestions
  - EDA suggestions
  - dashboard advice
- 

## 9. Knowledge Base Design

Each domain KB must include 4 types of documents:

1. Derived features
2. Statistical transformations
3. Domain KPIs
4. Modeling considerations

Important rule:

Store playbooks, not textbooks.

Bad:

“Introduction to statistics”

Good:

“How to create customer lifetime value feature”

---

## 10. Data Flow

CSV Upload



Profiler



Domain Classifier



Router



Retrieve domain knowledge



LLM Feature Agent



Feature Plan JSON

---

## 11. Performance Requirements

| Metric  | Target    |
|---------|-----------|
| Latency | < 15 sec  |
| Memory  | < 6GB RAM |
| GPU     | Optional  |
| Offline | Required  |

---

## 12. Failure Handling

The system must handle:

## Unknown Domain

Fallback → “general analytics” knowledge base

## Low Confidence Classification

Return top 2 domains and generate hybrid features.

## Small Dataset (< 50 rows)

Avoid aggressive feature engineering.

---

## 13. Evaluation Metrics

We measure success by:

1. Feature usefulness (human DS rating)
  2. Model improvement after features
  3. Relevance to domain
  4. Hallucination rate
- 

## 14. Out of Scope

This module will NOT:

- train models
- visualize charts
- answer chat questions
- clean data

Only feature engineering intelligence.

---

## 15. Tech Stack

Component

Tool

|            |                                      |
|------------|--------------------------------------|
| Embeddings | sentence-transformers<br>(bge-small) |
| Vector DB  | ChromaDB                             |
| LLM        | Llama 3 / Mistral via Ollama         |
| Framework  | LangChain                            |
| Language   | Python                               |

---

## 16. Deliverables

The module is complete when:

- uploading a dataset triggers domain detection
  - correct KB is used
  - at least 5 meaningful features are generated
  - output JSON is valid
  - no column hallucinations occur
- 

## 17. Future Extensions (not in current milestone)

- automatic feature creation code generation
  - automatic feature importance validation
  - feature store
  - learning from user feedback
-

# Data Repair & Integrity Layer



# EVA System Module Specification

## Data Repair & Integrity Layer (DRIL)

---

### 1. Purpose

The **Data Repair & Integrity Layer (DRIL)** is responsible for preparing the dataset for reliable analysis while preserving the meaning of the real-world process the data represents.

This module does not simply “clean” data.

Its objective is to ensure that conclusions drawn later are not distorted by data quality issues.

The DRIL performs corrective actions only when they improve analytical validity and never alters the dataset without recording and justifying the change.

All operations performed by this module are documented in the **Data Integrity Record** section of the Global Analysis Ledger (GAL).

---

### 2. Design Philosophy

Data is imperfect because real-world processes are imperfect.

Missing values, noise, and imbalance are not just technical problems.  
They are **signals about reality**.

Therefore, EVA follows three rules:

1. Not all irregularities should be removed.
2. Some irregularities contain meaningful information.
3. Cleaning decisions must depend on the analysis objective.

The goal is not a “perfect dataset.”

The goal is a **trustworthy dataset**.

---

### 3. Inputs

The Data Repair & Integrity Layer reads from the Global Analysis Ledger:

- dataset identity (row meaning, domain, column roles)
- time behavior
- misinterpretation risks
- user intent and selected target
- data health warnings

It operates on the dataset after understanding but before feature engineering and hypothesis validation.

---

## 4. Core Responsibilities

The DRIL must:

1. Detect integrity issues
2. Evaluate impact on analysis
3. Choose appropriate correction strategies
4. Record reasoning and effects
5. Validate repaired dataset

The module must act conservatively when uncertainty is high.

---

## 5. Missing Value Handling

The module evaluates missingness using three questions:

- How much data is missing?
- Where is it missing?
- Does missingness itself carry meaning?

Possible strategies:

- retain and flag missingness
- conditional replacement
- statistical replacement
- removal (only when safe)

Important rule:

Columns critical to the user's objective must not be dropped without justification.

The module records:

- missingness pattern
  - chosen treatment
  - expected analytical impact
- 

## 6. Duplicate Detection

The module identifies:

- exact duplicates
- near duplicates
- repeated entity records

The decision depends on row meaning:

Transaction dataset → duplicates are likely errors

Time-series dataset → repeated entries may be valid

Each removal must include justification and a count of affected records.

---

## 7. Data Type Corrections

The module detects columns stored in incorrect formats, such as:

- numeric values stored as text
- dates stored as strings
- encoded categories

Corrections are performed only if interpretation is unambiguous.

Ambiguous cases are flagged, not automatically changed.

---

## 8. Outlier & Noise Handling

The module evaluates whether extreme values represent:

- measurement errors
- rare but real events
- important anomalies

Possible actions:



- keep unchanged
- cap values
- transform
- flag for analysis
- remove (last resort)

The chosen action must consider the user's goal.

Example:

Fraud detection → keep extreme values

Average estimation → may cap extreme values

All decisions must be logged.

---

## 9. Class Imbalance Assessment

If a target variable exists, the module evaluates class distribution.

The module determines:

- severity of imbalance
- risk to interpretation
- necessity of correction

Possible actions include:

- no change
- weighting recommendation
- balanced sampling recommendation

Important:

The module does not optimize modeling performance.

It protects analytical validity.

---

## 10. Leakage Detection

The module must detect columns that accidentally reveal the outcome.

Examples:

- post-event timestamps
- calculated totals including the target
- encoded outcome labels

These columns are marked as **restricted features** in the Global Analysis Ledger and cannot be used in later stages.

---

## 11. Early Diagnostic Visualization (Internal)

To validate repairs, the module performs diagnostic inspections such as:

- distribution comparisons before vs after repair
- missingness pattern checks
- class balance checks

These visuals are internal verification tools and not user-facing insights.

They ensure repair decisions did not distort the dataset.

---

## 12. Validation Step

After all repairs, the module performs a final verification:

- dataset still represents original process
- target variable remains meaningful
- no new inconsistencies introduced
- column roles remain valid

Only after validation may the dataset move forward.

---

## 13. Output Written to Global Analysis Ledger

The module writes the **Data Integrity Record**, including:

Problem Identified

What issue existed

Impact Assessment

Why it mattered

Action Taken

Correction strategy

Alternatives Considered

Other possible actions

Effect on Dataset

Records changed or affected

Confidence Level

Certainty of correctness

---

## 14. Operational Rules

1. No silent modifications are allowed.
  2. Every change must include reasoning.
  3. The original dataset state must remain reproducible.
  4. Repair decisions must respect user intent.
  5. Downstream modules must read the integrity record before operating.
- 

## 15. Why This Module Is Critical

Most automated pipelines treat cleaning as a preprocessing step.

EVA treats it as **evidence preservation**.

Incorrect cleaning can:

- remove rare but important cases
- bias outcomes
- invalidate conclusions

The Data Repair & Integrity Layer ensures that later insights reflect reality rather than artifacts of bad data preparation.

This module protects the credibility of every recommendation EVA produces.

# Visualization Planner & Executor



# EVA System Module Specification

## Visualization Planner & Executor (VPE)

---

### 1. Purpose

The **Visualization Planner & Executor (VPE)** module is responsible for converting analytical understanding into meaningful visual representations.

The module does not generate charts arbitrarily.

Its role is to determine *which visual evidence a human needs to see* in order to understand the dataset and trust the conclusions.

Visualization in EVA serves a cognitive function:  
to validate observations, evaluate hypotheses, and communicate findings.

This module operates after Feature Reasoning and Investigation & Hypothesis generation.

---

### 2. Design Philosophy

Charts are not decoration.

Charts are arguments supported by evidence.

Typical systems:

data → charts → user interprets

EVA:

question → evidence → visualization → understanding

Every visualization must answer a specific question recorded in the Global Analysis Ledger (GAL).

If a chart does not answer a question, it must not be created.

---

### 3. Two-Stage Architecture

The module is divided into two independent components:

1. Visualization Planner (reasoning)
2. Visualization Executor (rendering)

Separating these ensures that visualization decisions are based on analytical need rather than automatic plotting.

---

## **PART A — Visualization Planner**

### **4. Responsibilities**

The Visualization Planner determines:

- what must be visualized
- why it must be visualized
- what comparison is important
- who the audience is

It reads from the Global Analysis Ledger:

- dataset identity
  - user intent
  - exploratory findings
  - hypotheses
  - engineered features
- 

### **5. Visualization Question Generation**

For each major finding or hypothesis, the planner creates visualization questions.

Examples:

- Is the relationship between two variables consistent?
- Does the outcome vary across groups?
- Is the trend stable over time?
- Are anomalies genuine or noise?
- Do segments behave differently?

Each question is stored in the ledger before any chart is created.

---

## 6. Visualization Type Selection

The planner selects a visualization strategy based on the analytical question and data type.

The decision depends on:

- column roles
- time behavior
- stakeholder type
- hypothesis being tested

Examples:

Comparing categories → group comparison visuals

Relationship between variables → relationship visuals

Time behavior → temporal visuals

Distribution understanding → distribution visuals

Segment differences → segment comparison visuals

The module records the reasoning behind each choice.

---

## 7. Audience Adaptation

Visualization complexity must match the stakeholder:

Student → educational clarity

Business owner → actionable simplicity

Analyst → detailed evidence

Researcher → rigorous comparison

The planner determines the required level of detail and annotation.

---

# PART B — Visualization Executor

## 8. Responsibilities

The Visualization Executor generates visual outputs based strictly on the planner's instructions.

It must not independently decide what to visualize.

Each visualization includes:

- title explaining the question
- annotation explaining insight
- reference to hypothesis or finding

Charts are evidence displays, not exploratory artifacts.

---

## 9. Visualization Validation

After generation, the module evaluates whether the visualization actually answers the intended question.

A visualization is rejected if:

- it is ambiguous
- it contradicts the underlying data
- it does not clarify the hypothesis

Rejected visuals are not shown to the user.

---

## 10. Output Written to Global Analysis Ledger

For each visualization, the module records:

Visualization Purpose

What question it answers

Related Finding

Which observation it supports

Related Hypothesis

What explanation it evaluates

Interpretation

What a user should learn

Confidence

Reminder that visuals support but do not prove causation

---

## 11. Operational Rules



1. No visualization may be created without a question.
  2. Visualizations must reference findings or hypotheses.
  3. Visualizations must not exaggerate effects.
  4. Visualizations must be understandable without technical knowledge.
  5. The executor cannot override the planner.
- 

## 12. Relationship to Other Modules

Investigation & Hypothesis Engine

→ determines what needs confirmation

Feature Intelligence Engine

→ provides meaningful variables to display

Analytical Dashboard Composer

→ selects which visualizations become permanent views

Report Generator

→ embeds selected visualizations into the final narrative

---

## 13. Why This Module Is Critical

Most automated systems produce many charts but little understanding.

EVA produces fewer charts, but each one has a purpose.

Visualization becomes:

not “showing data”

but

**showing evidence**

This module allows EVA to demonstrate reasoning rather than simply presenting statistics.

# Analytical Dashboard Composer



# EVA System Module Specification

## Analytical Dashboard Composer (ADC)

---

### 1. Purpose

The **Analytical Dashboard Composer (ADC)** module organizes EVA's findings into a persistent decision interface.

After analysis, hypotheses, and visualization validation are complete, the user should not need to repeatedly interpret individual charts. The dashboard presents the most important information in a structured and continuously understandable form.

The dashboard is not a collection of visualizations.  
It is a curated decision workspace.

The ADC determines which insights deserve permanent visibility and how they should be organized for ongoing interpretation.

---

### 2. Design Philosophy

A visualization explains one idea.

A dashboard explains a situation.

EVA does not build dashboards to display data.  
EVA builds dashboards to support decisions.

Each dashboard element must answer one of three questions:

- What is happening?
- Why is it happening?
- What requires attention?

If an element does not support these questions, it must not appear.

---

### 3. Inputs

The module reads from the Global Analysis Ledger:

- user intent
- stakeholder type
- validated visualizations
- hypotheses and plausibility
- key findings
- feature reasoning
- detected anomalies

It does not access raw data directly.

---

## 4. Core Responsibilities

The Analytical Dashboard Composer must:

1. Identify key performance indicators (KPIs)
2. Select essential visualizations
3. Organize them into logical sections
4. Provide interpretation guidance
5. Highlight priority cases
6. Maintain contextual continuity

The module creates structure, not graphics.

---

## 5. KPI Selection

The module identifies metrics that best represent the system being analyzed.

KPIs must:

- relate to the user's decision goal
- be understandable without statistics knowledge
- reflect meaningful change

Examples:

Business dataset → retention rate, activity level

Healthcare dataset → risk group distribution

Education dataset → performance trends

The module must justify each KPI in the ledger.

---

## 6. Dashboard Sections

The dashboard is structured into four logical panels.

---

### A. System Overview

Purpose: Provide immediate situational awareness.

Contains:

- key metrics
- major trends
- high-level summary

Answers:

“What is currently happening?”

---

### B. Drivers & Causes

Purpose: Explain important patterns.

Contains:

- selected explanatory visualizations
- linked hypotheses
- important feature effects

Answers:

“Why is this happening?”

---

### C. Risk & Alerts

Purpose: Surface urgent attention areas.

Contains:

- anomalies
- high-risk groups
- unusual observations

Answers:

“What needs attention now?”

---

## **D. Action Guidance**

Purpose: Support decision making.

Contains:

- recommended actions
- affected segments
- expected outcomes
- confidence level

Answers:

“What should be done?”

---

## **7. Stakeholder Adaptation**

The dashboard adapts based on the user type.

Student

→ explanatory emphasis

Business owner

→ decision emphasis

Researcher

→ evidence emphasis

Manager

→ summary emphasis

The module adjusts complexity, terminology, and number of elements.

---

## **8. Continuity Handling**

If the user reanalyzes updated data, the dashboard must support comparison.

The module identifies:

- changes since last session
- new risks
- improving or worsening indicators

The dashboard therefore becomes a monitoring interface rather than a static report.

---

## 9. Output Written to Global Analysis Ledger

The module records:

Selected KPIs  
Why they matter

Chosen Visualizations  
Why they were included

Highlighted Risks  
Reason for importance

Recommended Actions  
Link to supporting evidence

Audience Level  
Dashboard complexity

---

## 10. Operational Rules

1. Only validated visualizations may appear.
  2. Every dashboard element must map to a decision.
  3. Dashboard must remain understandable without technical training.
  4. No raw statistical tables should be shown by default.
  5. Dashboard must prioritize clarity over completeness.
- 

## 11. Relationship to Other Modules

Visualization Planner  
→ supplies candidate visuals

Investigation & Hypothesis Engine  
→ supplies explanations

Decision Guidance  
→ supplies recommended actions

Report Generator

→ converts dashboard insights into narrative documentation

---

## 12. Why This Module Is Critical

Without a dashboard, users receive analysis.

With a dashboard, users receive **awareness**.

Analysis answers questions once.

A dashboard keeps answering them continuously.

The Analytical Dashboard Composer turns EVA from a session-based tool into an ongoing analytical assistant.



# Report Generator



# EVA System Module Specification

## Report Generator (RG)

---

### 1. Purpose

The **Report Generator (RG)** module converts the complete demonstrated reasoning of EVA into a structured, human-readable analytical document.

The report is not a raw data export and not a collection of charts.  
It is a narrative explanation of:

- what the dataset represents
- what was discovered
- why it matters
- what should be done

The report allows a user to communicate results without needing to understand the internal analysis process.

The module produces a presentation-ready explanation derived entirely from the Global Analysis Ledger (GAL).

---

### 2. Design Philosophy

Data analysis is only valuable if it can be communicated.

Most users do not need more numbers.

They need a story they can explain confidently to another person.

The Report Generator therefore performs **translation**, not computation.

It translates:

technical reasoning → understandable narrative

The report must remain truthful to the analysis while remaining understandable to a non-technical reader.

---

### 3. Inputs

The module reads exclusively from the Global Analysis Ledger:

- dataset identity
- user intent
- data integrity record
- exploratory findings
- hypotheses
- feature reasoning
- validated visualizations
- dashboard selections
- recommendations

It must not perform new analysis.

---

### 4. Core Responsibilities

The Report Generator must:

1. Construct a coherent narrative
  2. Explain analysis decisions
  3. Present evidence clearly
  4. Communicate uncertainty
  5. Provide actionable conclusions
  6. Adapt language to audience
- 

### 5. Report Structure

The report follows a consistent structure.

---

#### A. Executive Summary

A short overview answering:

- What was analyzed
- What was discovered
- What it means

This section must be readable in under two minutes.

---

## **B. Dataset Understanding**

Explains:

- what the dataset represents
- what each record means
- domain context

This builds trust that the analysis is grounded in reality.

---

## **C. Data Preparation**

Summarizes the Data Repair & Integrity Layer:

- issues detected
- actions taken
- impact on reliability

Purpose: reassure the reader that conclusions are based on trustworthy data.

---

## **D. Key Findings**

Presents important observations:

- major patterns
- group differences
- trends

No technical statistics should be required to understand this section.

---

## **E. Explanations & Hypotheses**

Summarizes Investigation & Hypothesis Engine results:

- possible causes
- supporting evidence
- uncertainty

Important rule:

The report must clearly distinguish:  
observation vs explanation.

---

## **F. Visual Evidence**

Includes selected validated visualizations with explanations:

Each visual must include:

- what it shows
- why it matters
- how it supports a conclusion

Charts must never appear without interpretation.

---

## **G. Recommendations**

Provides clear guidance:

- suggested actions
- affected groups
- urgency level
- expected impact

Recommendations must reference findings and evidence.

---

## **H. Limitations & Confidence**

The report must explicitly state:

- uncertainties
- possible data weaknesses
- where caution is needed

This increases trust and prevents overclaiming.

---

## **6. Audience Adaptation**

The module adjusts tone and detail based on stakeholder:

Student

→ educational explanation

Business owner

→ action-focused summary

Manager

→ concise overview

Researcher

→ structured reasoning

The same analysis produces different narratives without changing conclusions.

---

## **7. Output Characteristics**

The report must be:

- readable without technical knowledge
- logically structured
- reproducible from ledger
- evidence-backed
- honest about uncertainty

The report must not include raw statistical tables unless explicitly requested.

---

## **8. Output Written to Global Analysis Ledger**

The module records:

Generated Narrative

Final explanation

Referenced Evidence

Ledger sources used

Included Visualizations

What was shown

Communicated Recommendations

Final guidance

This allows regeneration of the report later.

---

## 9. Operational Rules

1. No new analysis may occur here.
  2. Every claim must trace to ledger evidence.
  3. The report must not exaggerate certainty.
  4. Hypotheses must not be presented as proven causes.
  5. The report must be understandable to a non-technical reader.
- 

## 10. Relationship to Other Modules

Global Analysis Ledger

→ source of truth

Visualization Planner

→ provides visual explanations

Analytical Dashboard Composer

→ provides key insights

Investigation & Hypothesis Engine

→ provides reasoning

Decision Guidance

→ provides action steps

---

## 11. Why This Module Is Critical

Without a report, the user must interpret results themselves.

With a report, the user can **communicate decisions**.

EVA therefore does not stop at analysis.

It finishes the last step of real data work:

explaining the answer to another human.

The Report Generator is what makes EVA usable in education, business, and research environments.

